Assignment Beta – Question 2
LoRA vs QLoRA vs DoRA vs AdaLoRA for function calling

Syed Ukkashah · 23K-0055 · Notebook: Assignment_Beta_Question_no_2_23K-0055.ipynb

Section 1 – Theory

(a) Two matrices are added: B is 4096 × r and A is r × 4096. Parameters = 4096·16 + 16·4096 = 2 · 4096 · 16 = 131,072.

(b) W_o has 4096^2 = 16,777,216 parameters, so 131,072 / 16,777,216 = 0.78% (exactly 2r/d = 32/4096).

(c) With B = 0 the update ΔW = BA is zero at step 0, so the adapted model starts exactly at the pretrained model. A must be random so that learning can start: $\partial L/\partial B = (\partial L/\partial h)(Ax)^T$ is non-zero only because A ≠ 0, and after one step B is non-zero and A starts receiving gradient too. If both were random, BA would be random noise added to every adapted layer before any learning. The first forward pass would already be a damaged model (worse instruction following and JSON formatting), and the first gradients would be spent undoing that noise instead of learning the task. If both were zero, neither would ever receive a gradient.

(d) What each method fixes that plain LoRA does not:

- **QLoRA –** Plain LoRA still holds the frozen base in 16-bit, which dominates memory. QLoRA stores the base in 4-bit NormalFloat (NF4), whose 16 levels sit at the quantiles of a normal distribution, matching how pretrained weights are distributed. Double quantisation also compresses the per-block scales. Paged optimizers move optimizer state to CPU RAM during memory spikes, so a long batch slows training down instead of crashing it.
- **DoRA –** Writes each weight column as magnitude × direction, $W = m \cdot V/\|V\|$. LoRA updates only the direction ($V = W_o + BA$), while the magnitude vector m is trained separately. LoRA tends to change both together; separating them makes DoRA behave more like full fine-tuning, especially at low rank, at the cost of recomputing column norms.
- **AdaLoRA –** Plain LoRA gives every matrix the same rank. AdaLoRA writes each update in SVD form PΛQ, scores every singular value by importance (smoothed $|w \cdot \partial L/\partial w|$ plus an uncertainty term) and prunes the least important ones on a schedule. A global rank budget (init_r → target_r) therefore flows to the matrices that need it.

Section 2 – Experiment and results

Setup. Base model Qwen2.5-1.5B-Instruct. Data: glaive-function-calling-v2, shuffled with seed 42, keeping examples whose first assistant turn is a function call. 2,000 examples: 1,800 train and 200 held-out test, saved to disk and never used in training. Identical for every run: 1 epoch, learning rate 2e-4 with cosine schedule and 3% warm-up, effective batch 16 (2 × 8 accumulation), paged AdamW, fp16 compute, adapters on q/k/v/o projections, α = 16, dropout 0.05, loss on answer tokens only. Evaluation: greedy decoding, up to 160 new tokens, on one Colab T4 (16 GB). Only the adaptation method changes between rows.

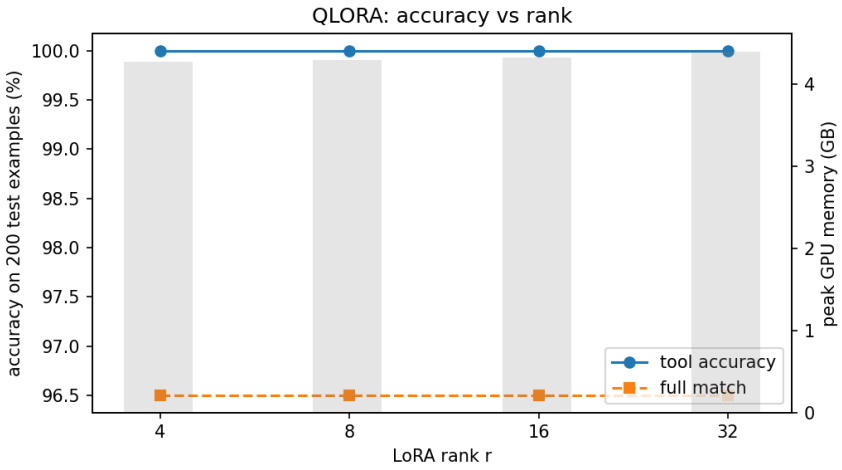
Run	Trainable params (% of model)	Peak GPU mem (GB)	Train (min)	Valid JSON	Tool acc.	Full match	Gen. tok/s
Base (no fine-tuning)	0 (0.000%)	3.09	0.0	200/200	100.0%	95.0%	102
LoRA r=8 (fp16)	2,179,072 (0.141%)	5.20	7.0	200/200	100.0%	95.5%	98
QLoRA r=8 (NF4)	2,179,072 (0.141%)	4.29	10.0	200/200	100.0%	96.5%	71
DoRA r=8 (fp16)	2,279,424 (0.147%)	5.20	10.0	200/200	100.0%	95.5%	65
AdaLoRA 12→8 (fp16)	3,270,064 (0.211%)	5.27	9.2	200/200	100.0%	95.0%	93

Every run produced valid JSON for all 200 test prompts and picked the right function every time. Full match separates the runs by at most 3 examples. The best run (QLoRA, 96.5%) has a 95% Wilson interval of [93.0%, 98.3%], which contains every other row. Against the base model it fixes 5 examples and breaks 2 (exact McNemar p = 0.45). On accuracy the methods are tied; they differ in cost. (AdaLoRA's logged training loss, 3.59, includes PEFT's orthogonality penalty and cannot be compared with the others' ≈0.008.)

Section 3 – Is a bigger rank actually better?

QLoRA came out on top (by full match), so it was retrained at r = 4, 16 and 32 with everything else unchanged.

Rank	Trainable params	Peak mem (GB)	Train (min)	Tool acc.	Full match
r = 4	1,089,536	4.27	9.5	100.0%	96.5%
r = 8	2,179,072	4.29	10.0	100.0%	96.5%
r = 16	4,358,144	4.32	9.6	100.0%	96.5%
r = 32	8,716,288	4.39	9.5	100.0%	96.5%



Conclusion. No. The graph is flat: tool accuracy is 100% and full match is 96.5% at every rank, while trainable parameters grow 8× (1.09 M → 8.72 M) and peak memory creeps up from 4.27 to 4.39 GB. r = 4 and r = 8 produced character-for-character identical outputs on 200/200 test prompts. On this task, doubling the rank only spends more memory (and adapter storage) for the same result, so r = 4 is enough. The likely reason is that the task needs a very small change of behaviour (the base model is already at 95%), so even rank 4 has more capacity than is used.

Section 4 – Report

4.1 Your call: what ships on Monday

Pick: LoRA (fp16 base), r = 8 as tested; r = 4 is a cheap follow-up, given the flat rank curve.

Accuracy cannot decide, because the four methods are statistically tied. So the decision comes down to the other three criteria:

- **Speed:** LoRA trains fastest (7.0 min against 9.2–10.0 min for the others) and generates the fastest while unmerged (98 tok/s, close to the base model's 102). QLoRA generates at 71 tok/s because every forward pass de-quantises 4-bit weights. DoRA is slowest at 65 tok/s because it recomputes column norms for the magnitude/direction split on every call.
- **Memory:** QLoRA's memory saving is small at this size: 4.29 GB against 5.20 GB peak. Only the linear layers become 4-bit. prepare_model_for_kbit_training upcasts the remaining layers (the 233 M-parameter

embedding, the norms) to fp32, and activations for 768-token batches cost the same either way. Every option fits a 16 GB T4 three times over. QLoRA would win for a 7 B+ base on the same card.

- **Merging (the hint):** A LoRA adapter trained on fp16 weights merges exactly into those weights: $W = W_o + BA$ has no extra latency and no change in answers. DoRA also merges exactly ($m \cdot V/\|V\|$ folds into one matrix), but when kept unmerged for multi-adapter serving it is the slowest. QLoRA is the awkward one. Its adapter was trained against NF4-de-quantised weights, so merging it into the fp16 original gives weights the adapter never saw. Merging into 4-bit means re-quantising and adding rounding error. Our merge check shows this: QLoRA merged into fp16 generated at 138 tok/s but full match fell from 96.5% to 95.0% (3 answers changed).

LoRA gives the same accuracy, the fastest training and generation, and an exact, lossless merge (or near-zero overhead unmerged, which matters for 4.3). It is the least sophisticated option, and the most practical one.

4.2 Where it still breaks

The five failures below come from the best run (QLoRA r = 8). It has 7 misses in total; the other two are a dataset label quirk (the label keeps "30th April", the model normalised it to "2022-04-30") and a second recipe example that drops one of two keywords.

Customer message	Correct call	Model produced	Label
"…I can't remember the full title. I think it had the word "Sun" in it and it was a mystery novel."	search_books(title="Sun", genre="mystery")	search_books(title="", author="", genre="mystery")	Something else: required value dropped, empty unrequested arg
"…a movie… I know it has the word "star" in it and it's a sci-fi movie."	search_movies(title="star", genre=["sci-fi"])	search_movies(title="", genre=["sci-fi"], keyword="star")	Made-up argument ("keyword" is not in the schema)
"I am looking for a laptop under $1000."	search_product(query="laptop", category="Electronics", price_range={0,1000})	search_product(query="", category="laptop", price_range={0,1000})	Something else: value put in the wrong argument
"I'm looking for a quick Italian recipe that uses chicken and tomatoes."	search_recipes(ingredients=[chicken, tomatoes], cuisine="Italian", max_prep_time=30)	search_recipes(ingredients=[chicken, tomatoes], cuisine="Italian")	Something else: vague word ("quick") not turned into an argument
"…create an event for my birthday party. It's on 25th of next month from 7pm to 10pm…at my house."	create_event(title="My Birthday Party", start="2022-05-25T19:00", end="…22:00", location="My house")	create_event(title="Birthday Party", start="2023-04-25T19:00Z", end="…22:00Z", location="My House")	Something else: ungrounded date (no "today" in the prompt)

Pattern. No wrong tools, no broken JSON, no refusals. Every failure is about filling in arguments, and they all start where the user is vague: "has the word Sun in it", "quick", "next month". In those cases the model either leaves a required field empty and moves the value elsewhere, even inventing a field to hold it (keyword), or it guesses. Choosing the tool is solved; grounding vague values into arguments is not. Two of the "errors" are arguable: the label's own date (2022-05) is just as much a guess as the model's, which suggests the fix is to put today's date in the system prompt or add a date tool. Also, the test set only contains prompts that should trigger a call, so "avoids unnecessary actions" can only be judged from the extra arguments seen here. A production evaluation needs no-call examples too.

4.3 The agentic angle

A router, a refund agent and a shipping agent can share one frozen Qwen2.5-1.5B in GPU memory (3.1 GB in fp16), each with its own LoRA adapter of about 9 MB (2.2 M parameters). Three separate fine-tuned models would cost 3 × 3.1 GB of GPU memory, three deployments to keep up to date, and three full training runs. With adapters, each agent costs about 7 minutes of T4 training and a few MB of storage. Multi-LoRA servers such as vLLM or LoRAX (or PEFT's set_adapter) can serve requests for different adapters in the same batch on the same base, so adding a fourth agent costs almost nothing.

Risk: A request can be served by the wrong or stale adapter during a hot-swap. If adapter v2 is loaded while a multi-turn refund conversation is in flight, or adapter slots are evicted and reloaded under load, a turn can be generated by the shipping adapter or by a half-updated refund adapter. That agent may then call a tool it should never call, such as issuing a refund from the shipping flow. Nothing crashes, so the failure is silent. Mitigations: pin an adapter ID and version per conversation, swap adapters atomically (load the new one, then switch traffic), and check every emitted tool call against that agent's allowed-tool list before running it.

4.4 What surprised me

- **The base model was nearly perfect.** I expected the untuned model to fail on format, but it already produced valid JSON for 200/200 prompts, picked the right tool 100% of the time and fully matched 95%. Clear instructions plus the schema in the prompt did most of the work, and fine-tuning moved the needle by 3 examples.
- **Near-zero loss, almost no gain.** LoRA's training loss reached about 0.008, which looks like memorising the format, yet test accuracy barely changed, because the model was already producing that format.
- **Rank changed nothing.** r = 4 and r = 8 gave identical outputs on 200/200 prompts, and r = 32 trained 8× the parameters for the same score. I expected at least a small upward trend.
- **QLoRA's memory saving was tiny at 1.5 B.** QLoRA saved only 0.9 GB here but trained 41% slower and generated 27% slower than LoRA. Its advantage depends heavily on model size.
- **The metric is partly measuring the dataset.** Some "errors" are the model being more useful than the label (an ISO date instead of "30th April"), and some labels invent values the user never gave (category "Electronics", max_prep_time 30). Exact-match scoring punishes reasonable answers, so on this dataset a 1–2 point difference says more about the labels than about the method.